Programación Web - II

reto del PATCH de stock | Manizales, Caldas

INGENIERIA EN INFORMATICA

NICOLAS CORDOBA RAMIREZ

WEEK 5

RETO : agrega el endpoint PATCH /api/productos/:id/stock que recibe {"delta": -3} y ajusta el stock. Regla de negocio en el service: si el resultado quedaría negativo, lanzar conflicto ('Stock insuficiente'). Este endpoint es un adelanto directo de "descontar stock al crear pedido" de la semana 7.

```
cordo@NicoCR MINGW64 ~/Desktop/Web-II/campuseats
$ curl -i -X PATCH http://localhost:3000/api/productos/1/stock \
  -H "Content-Type: application/json" \
  -d "{\"delta\": -3}"
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 63
ETag: W/"3f-BDcz6mQkK0Nh5cTN0kupEkXqX1U"
Date: Fri, 30 Oct 2026 00:43:48 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"id":1,"nombre":"Almuerzo del día","precio":12000,"stock":22}
cordo@NicoCR MINGW64 ~/Desktop/Web-II/campuseats
```

src/controllers/productos.controller.js

```js
// src/controllers/productos.controller.js
import * as servicio from '../services/productos.service.js';

export function listar(req, res) {
  const productos = servicio.listar({
    disponibles: req.query.disponibles === 'true',
  });
  res.json(productos);
}

export function obtener(req, res) {
  const producto = servicio.buscarPorId(Number(req.params.id));
  if (!producto) return res.status(404).json({ error: 'Producto no encontrado' });
  res.json(producto);
}

export function crear(req, res) {
  try {
    const nuevo = servicio.crear(req.body);
    res.status(201).json(nuevo);
  } catch (error) {
    if (error.message === 'DATOS_INVALIDOS') {
      return res.status(400).json({ error: 'Nombre y precio (número >= 0) son oblig
    }
    if (error.message === 'NOMBRE_DUPLICADO') {
      return res.status(409).json({ error: 'Ya existe un producto con ese nombre' }
    }
    res.status(500).json({ error: 'Error interno' });
  }
}

export function actualizar(req, res) {
```

```
  try {
    const producto = servicio.actualizar(Number(req.params.id), req.body);
    if (!producto) return res.status(404).json({ error: 'Producto no encontrado' });
    res.json(producto);
  } catch {
    res.status(400).json({ error: 'Datos inválidos' });
  }
}

export function eliminar(req, res) {
  const eliminado = servicio.eliminar(Number(req.params.id));
  if (!eliminado) return res.status(404).json({ error: 'Producto no encontrado' });
  res.status(204).end();
}

// 👉 RETO: Controlador para el PATCH de stock
export function ajustarStock(req, res) {
  try {
    const { id } = req.params;
    const { delta } = req.body;

    if (delta === undefined || typeof delta !== 'number') {
      return res.status(400).json({ error: 'El campo delta es obligatorio y debe ser un número
    }

    const productoActualizado = servicio.ajustarStock(id, delta);
    return res.json(productoActualizado);
  } catch (error) {
    return res.status(error.status || 500).json({ error: error.error || error.message || 'Erro
  }
}
```

src/routes/productos.routes.js

```js
// src/routes/productos.routes.js
import { Router } from 'express';
import * as controlador from '../controllers/productos.controller.js';

const router = Router();

router.get('/', controlador.listar);
router.get('/:id', controlador.obtener);
router.post('/', controlador.crear);
router.put('/:id', controlador.actualizar);
router.delete('/:id', controlador.eliminar);

// 👉 RETO: Ruta PATCH para ajustar el stock
router.patch('/:id/stock', controlador.ajustarStock);

export default router;
```

src/services/productos.service.js

```js
// src/services/productos.service.js
let productos = [
  { id: 1, nombre: 'Almuerzo del día', precio: 12000, stock: 25 },
  { id: 2, nombre: 'Jugo natural', precio: 4000, stock: 50 },
  { id: 3, nombre: 'Café campesino', precio: 2500, stock: 100 },
];
let siguienteId = 4;

export function listar({ disponibles } = {}) {
  if (disponibles) return productos.filter((p) => p.stock > 0);
  return productos;
}

export function buscarPorId(id) {
  return productos.find((p) => p.id === id) ?? null;
}

export function crear({ nombre, precio, stock = 0 }) {
  if (!nombre || typeof precio !== 'number') {
    throw new Error('DATOS_INVALIDOS');
  }
  if (precio < 0) throw new Error('DATOS_INVALIDOS');
  if (productos.some((p) => p.nombre === nombre)) {
    throw new Error('NOMBRE_DUPLICADO');
  }
  const nuevo = { id: siguienteId++, nombre, precio, stock };
  productos.push(nuevo);
  return nuevo;
}

export function actualizar(id, cambios) {
  const producto = buscarPorId(id);
```

```
  if (!producto) return null;
  if (cambios.precio !== undefined && cambios.precio < 0) {
    throw new Error('DATOS_INVALIDOS');
  }
  if (cambios.nombre !== undefined) producto.nombre = cambios.nombre;
  if (cambios.precio !== undefined) producto.precio = cambios.precio;
  if (cambios.stock !== undefined) producto.stock = cambios.stock;
  return producto;
}

export function eliminar(id) {
  const existia = productos.some((p) => p.id === id);
  productos = productos.filter((p) => p.id !== id);
  return existia;
}

// 👉 RETO: Ajustar stock con delta y regla de negocio (si queda negativo, lanza conflicto)
export function ajustarStock(id, delta) {
  const producto = buscarPorId(Number(id));
  if (!producto) {
    throw { status: 404, error: 'Producto no encontrado' };
  }

  const nuevoStock = producto.stock + delta;

  if (nuevoStock < 0) {
    throw { status: 409, error: 'Stock insuficiente' };
  }

  producto.stock = nuevoStock;
  return producto;
}
```